June 2026

OpenWorld: Training-Free Symbolic World Models with Verified Code Dynamics, Tunable Moral Configurations, and Agents-as-a-Judge

Jim Schwoebel[1,*]

[1] Quome

The author is a member of the *Task Force for AI Agents in Healthcare*.

Abstract

World models built on learned neural latent dynamics achieve striking results but remain data-hungry, stochastic, and opaque, and their values are frozen into weights. Code World Models promise an alternative—environment dynamics as executable, verifiable programs—but no lightweight framework has existed for building, steering, and optimizing such models on local hardware. We present **OpenWorld**, a zero-dependency Python framework in which a local LLM plans, writes, and *verifies* the executable dynamics of a declared world; objectives remain open, editable artifacts weighted by inference-time moral dials; an automated tuner searches world, policy, and moral configurations jointly; and an agent-as-a-judge selects among candidate behaviors. Across 40 seed-fixed experiments on local 7B/3B/1.5B models we find: verified synthesized dynamics are exact on 24/24 twenty-step rollouts across three ground-truth worlds (95% CI [0.86, 1.00]) while direct LLM next-state prediction (a learned-dynamics proxy) completes 0/24; a genuinely *trained* baseline—an MLP given 10,000 environment transitions—reaches only 50% exact accuracy on branch-covering probes and 0% at 10× out-of-distribution scale, where the synthesized program, built from rule text alone with zero transitions, scores 100%; rollouts run ∼47,087× faster than LLM prediction; verification gates raise the semantic correctness of a weak generator's accepted dynamics from 0.46 to 0.70, with an ablation separating the filter (quality) from the repair loop (throughput); the synthesis result replicates across model families (Llama-3.1-8B: 0.85 probe accuracy, Gemma-2-9B: 1.00, both 100% verified acceptance at 4–9 s per world); and on a 20-task program-repair benchmark with paired controls pooled over 120 rounds, judge selection solves 85% against 78% for random selection over the same candidates—a margin that sharpened with sample size but remains marginal (exact McNemar $p = 0.078$)—while picking a passing candidate ∼85% of the time under either candidate order, near the 91% oracle ceiling. Depth-3 lookahead planning through the synthesized simulator matches planning through ground truth exactly and beats reactive and random baselines, while planning through the LLM proxy scores worse than no model at all; a complexity stress test locates the 7B synthesis cliff near eight interacting rules; seeded-RNG synthesis extends the paradigm to stochastic worlds within 1.2 points of the declared distribution; and ensemble disagreement across independently synthesized programs detects every ground-truth error without an oracle. A moral-philosophy review round further broadens the value machinery beyond its implicit consequentialism: Rawlsian and lexicographic aggregators, deontological side constraints enforced as action vetoes (eliminating the impermissible region a weighted sum cannot exclude), and a moral parliament that hedges across ethical theories. The central result, within the symbolic-state regime these worlds inhabit: a training-free, verified, symbolic world model on consumer hardware eliminates compounding rollout error at zero training cost. The advantage is not merely an artifact of being handed the rules—on equal information (traces only, no rule text), code *induction* still extrapolates where learned baselines collapse out of distribution (0.43 vs 0.00), though the rules-vs-

traces gap (1.00 vs 0.43) measures what a specification genuinely buys. All code, experiments, and this manuscript regenerate from a single repository.

Keywords: world models; code world models; neuro-symbolic AI; program synthesis; value alignment; agents-as-a-judge; local LLMs.

Code & data: `github.com/jim-schwoebel/openworld`

Correspondence: jim@quome.com

* Corresponding author.

1 Introduction

A world model is an agent's internal simulator: a mechanism that predicts how the environment evolves under action, enabling planning "in imagination" rather than by trial in the world [7]. The dominant research line learns this simulator as a neural network over latent states—from the V–M–C triad of Ha and Schmidhuber [7] through the RSSM-based Dreamer family [8], value-equivalent planning in MuZero [20], autoregressive token models such as IRIS [12] and Δ-IRIS [13], diffusion dynamics in DIAMOND [2], and internet-scale generative environments in Genie [4]. These systems share three structural costs: they are *data-hungry* (millions of environment steps, or internet-scale video), *compounding* (every learned transition adds error that accumulates over a rollout), and *opaque* (dynamics and values alike live in continuous weights that cannot be inspected, edited, or audited).

Code World Models (CWMs) invert the bargain: represent dynamics as executable programs synthesized by an LLM, gaining verifiability, compute-cheap rollouts, and out-of-distribution generalization by construction [23, 6, 17, 9]. Yet CWM research has remained a set of bespoke systems aimed at particular benchmarks. There has been no general, lightweight framework that lets a practitioner *declare* a world, have a local model write and verify its dynamics, steer behavior through declared value weights, and optimize the whole configuration against a goal—the workflow that made supervised learning ubiquitous.

OpenWorld is that framework, and this paper benchmarks the paradigm it operationalizes against the learned-dynamics alternative. The framework contributes four mechanisms: (i) a *plan–generate–verify relay* in which a generator LLM writes transition code that must survive syntactic checks, sandboxed smoke-runs, declared invariants, and optionally a second-model critic before acceptance; (ii) *open value specification*—objectives are scoring functions weighted by tunable dials, so moral configuration is an editable artifact rather than a frozen weight, directly addressing the specification trap [21]; (iii) an *automated tuner* that searches world design, policy knobs, and moral dials jointly (random, local-refinement, and Optuna TPE [1] strategies); and (iv) *agents-as-a-judge* [26, 25]: an LLM judge that selects among candidate behaviors and grades trajectories against written rubrics.

A review of the world-model landscape reveals distinct strategy families with a recurring tension—fidelity and scale on one side; verifiability, sample efficiency, and steerability on the other (Table 1).

Contributions. Our contributions are as follows:

1. **The central result.** On three worlds with known ground truth, verified synthesized dynamics are exact on 24/24 20-step evaluation rollouts (95% CI [0.86, 1.00]) and answer 10×

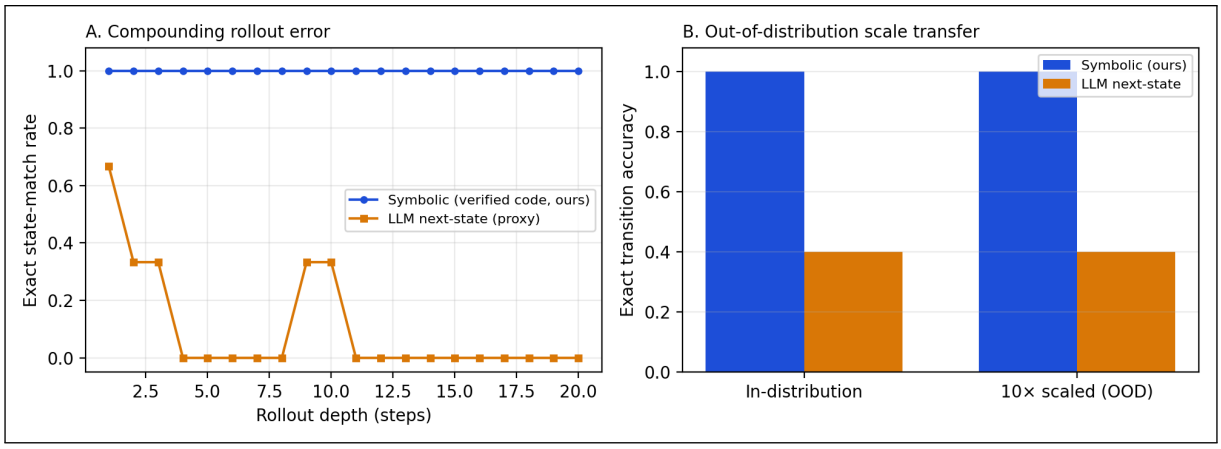

Figure 1: **The paper in one figure.** *A:* On a ground-truth-instrumented world, dynamics synthesized and verified once as code (blue) match the oracle exactly at every depth of a 20-step rollout, while per-step LLM next-state prediction (orange)—a structural stand-in for learned neural dynamics—first diverges at step 2.3 on average and never completes an exact rollout. *B:* The same synthesized program answers single-transition probes exactly at both nominal and 10× out-of-distribution magnitudes (100%), while the learned-style engine is wrong on more than half of probes at *both* scales. Programs encode rules, not statistics: the compounding-error failure mode of learned world models is eliminated by construction, at zero training cost.

out-of-distribution probes at 100%, while the LLM next-state proxy completes 0/24 rollouts and a trained MLP given 10,000 transitions scores 0% out of distribution—at a ∼47,087× rollout-speed disadvantage for the LLM engine.

2. **The system.** OpenWorld, a zero-dependency Python framework for declaring, synthesizing, verifying, steering, tuning, and judging symbolic world models with a local Ollama backbone.

3. **The benchmark and protocol.** Three ground-truth-instrumented worlds, a 20-task program-repair benchmark (the coding world), and 40 seed-fixed experiments with paired statistical tests, all regenerable from one repository.

4. **Measured guardrails.** Verification and judging are evaluated, not assumed: ablations isolate what each verification gate buys (+0.24 absolute probe accuracy over blind acceptance) and decompose the mechanism (the filter holds quality, the repair loop converts 62% acceptance into 100%); against paired controls on shared candidates pooled over 120 rounds, judge selection exceeds random selection (85% vs 78%, exact McNemar $p = 0.078$, marginal) and approaches the 91% oracle ceiling, with a position-bias audit showing order-sensitive choices but order-stable accuracy; judge rubric scores track a ground-truth objective at Spearman $\rho = 0.75$ ($p = 0.0057$, permutation test), directionally stable under rubric paraphrase ($\rho = 0.58$, $p = 0.0657$).

Positioning in the field. OpenWorld operationalizes the symbolic side of the learned-vs-symbolic divide: it is the CWM program [23, 9] packaged as general infrastructure, extended with the open-specification stance of the value-alignment literature [21, 11] and judge-based behavior selection [26]. We deliberately benchmark on consumer hardware with 7B/3B local models: the regime where learned world models are least accessible and training-free approaches matter most.

2 Problem Setting and Positioning

We target symbolic-state environments: worlds whose state is a JSON-serializable structure, whose action set is discrete, and whose dynamics follow declarable rules. This covers operational simulations (triage, negotiation, portfolios, sprints) and program repair, but not pixel-native domains; we return to this boundary in the limitations. The practitioner declares a world (state schema, actions, plain-language rules), and the framework must produce a faithful, fast, auditable simulator plus the machinery to steer and optimize behavior within it. The adversary here is not a security attacker but *model error itself*: hallucinated transitions, silently wrong code, reward misspecification, and value drift. Table 1 situates the approach against the principal world-model strategy families surveyed in the accompanying literature review.

System	Approach	Gap OpenWorld targets
World Models [7]	VAE + MDN-RNN latent dynamics	millions of env. steps; opaque latents
DreamerV3 [8]	RSSM, discrete latents, imagination	compounding rollout error; values in weights
MuZero [20]	value-equivalent latent planning	ungrounded states; incomplete context
IRIS / Δ-IRIS [12, 13]	VQ tokens + autoregressive transformer	token budgets; drift over horizon
DIAMOND [2]	diffusion dynamics in pixel space	GPU-heavy; no symbolic audit
Genie [4]	generative interactive video (11B)	frontier-scale compute; closed
NE-Dreamer [14]	decoder-free next-embedding prediction	still trained; latent, unauditable
WorldCoder / GIF-MCTS [23, 6]	LLM-synthesized code dynamics	bespoke pipelines, no reusable framework
PoE-World [17]	products of programmatic experts	per-domain expert engineering
NeSyS [15]	symbolic rules constrain neural logits	still requires fine-tuning data
OpenWorld (this work)	declared worlds; verified code dynamics; moral dials; tuner; judge	—

Table 1: Positioning against representative world-model strategies from the learned and symbolic families.

3 System Design

3.1 Overview and components

OpenWorld composes six unit-testable components: `World` (declared state ontology, actions, rules, and a pluggable transition engine); three interchangeable dynamics engines (`CodeTransition`: synthesized, verified, executable Python; `LLMTransition`: per-step LLM next-state prediction, our learned-style baseline; `FunctionTransition`: hand-written oracle); `Agent` (LLM planner or deterministic policy); `Objective+Dial` (open value specification); `Simulation` (scored trajectories); `Tuner` (configuration search); and `Judge` (candidate selection and rubric grading). The backbone

is any model served by Ollama, reached through the Python standard library; the framework has zero runtime dependencies.

3.2 A declarative world specification

A world declaration and its verified compilation

```
world = World(
    name="sprint",
    description="An engineering team working a sprint backlog.",
    initial_state={"backlog": 12, "shipped": 0, "bugs": 0, "debt": 0},
    actions=["ship", "fix", "refactor"],
    rules=["'ship' (when backlog > 0): backlog -1, shipped +1, debt +1, "
           "then bugs increase by debt // 4", "..."],
    llm=OllamaLLM(model="qwen2.5:7b"),
)
world.compile(                          # generator writes transition(state, action)
    critic=OllamaLLM(model="qwen2.5:3b"),  # optional second-model review
    invariants=[("counters never negative",
                 lambda s: all(v >= 0 for v in s.values()))],
)                                       # accepted code is a plain, editable artifact
```

Figure 2: The interface: a world is *declared*; its dynamics are synthesized and must survive syntax checks, sandboxed smoke-runs on every action, declared invariants, and an optional LLM critic, with verifier feedback looped back to the generator until acceptance.

3.3 The plan–generate–verify relay

`compile()` prompts a generator model with the world context and demands a single pure function `transition(state, action)`. Each candidate passes three gates: (1) *syntactic*—the code parses and defines the required function; (2) *behavioral*—it executes in a restricted sandbox (curated builtins; no imports or I/O; inputs deep-copied so candidate code can never mutate caller state) against every declared action and must satisfy the world's invariants; (3) *semantic* (optional)—a second model reviews the code against the written rules and returns `PASS` or concrete feedback. Any failure is appended to the next generation prompt; the loop is bounded and returns the full attempt history on failure. Accepted dynamics are plain source: they can be diffed against intent, unit-tested, edited, saved, and reloaded.

3.4 Open value specification and configuration search

Objectives are named scoring functions over (s, a, s') weighted by floats or `Dials`—tunable scalars adjustable mid-simulation. Trajectories record each objective's raw series, so dial sweeps trace true Pareto frontiers rather than scalarized aggregates. The `Tuner` treats world parameters, policy thresholds, and dials as one search space (dials are first-class: passed directly, tuned over their declared bounds), evaluates each candidate configuration by simulation, and supports broad random search, Optuna TPE [1] for expensive worlds, and hill-climbing refinement around incumbents.

3.5 Agents-as-a-judge

Following the agent-as-a-judge line [26, 25], a `Judge` wraps any backbone model with two operations: `choose(options, context)`—select among N candidate actions, patches, or plans—and `score_-`

`trajectory(trajectory, rubric)`—grade an episode 0–10 against a written rubric. Unparseable verdicts degrade to safe defaults. Judges plug into action selection (§5.5) and can serve as objectives where no programmatic score exists.

3.6 Implementation

Python (≥3.9), zero runtime dependencies (Optuna optional), ∼1,800 lines across 14 modules, 44 unit tests that run fully offline via a scripted mock backbone. Released with worked examples, four domain tutorials, the three instrumented worlds, the 20-task coding benchmark, and every experiment script used here.

4 Experimental Setup

Worlds with ground truth. Three deterministic oracle worlds instrument every dynamics comparison: *sprint* (backlog/debt/bug dynamics with an integer-division interaction), *orchard* (resource pool with per-agent tallies), and *triage* (queues, a clock, and deterioration events). Oracles are hand-written `FunctionTransition`s; synthesized and learned-style engines are compared against them exactly, including on a fixed probe suite that exercises clamping, empty-queue, and interaction branches.

The coding world. Program repair is the flagship use case: well-known, objectively scored, and natively symbolic [5]. Each of 20 tasks is a buggy Python function plus a hidden test suite spanning classic defect archetypes (off-by-one ranges and binary-search bounds, inverted comparisons, missing normalization, wrong base cases, unsorted medians, order-destroying dedup, early imbalance, whitespace splitting, accumulator resets, integer-division truncation, overlapping counts, dropped final runs, and degenerate-input handling). The world's transition *is* test execution: a submitted patch runs bit-exactly in a sandbox with a wall-clock alarm (a looping patch fails rather than hangs), and failing-test feedback enters the state for the next attempt.

Benchmark-scale repair (E28–E29). Beyond the single-function benchmark, two SWE-bench-style suites probe repair at module scale. Each instance carries a natural-language issue report (symptoms and a repro, never the fix), a buggy module (functions or a stateful class), two hidden suites (`fail_to_pass` exercising the defect, `pass_to_pass` guarding regressions), and an explicit world specification whose `submit_patch` dynamics run both suites bit-exactly; solving requires zero failures in *both*, so a fix that breaks a passing test does not count. The *atomic* suite (E28, $n = 6$) plants one issue-described defect per instance; the *staged* suite (E29, $n = 15$) plants a second, latent defect that surfaces only as a new failing test once the issue-visible defect is fixed. Both run a paired ablation across the `qwen2.5` ladder (1.5B/3B/7B): the same model *single-shot* (one completion from issue + module) versus *in-world* (up to 4 `submit_patch` steps; the first prompt is identical to single-shot, and exact failing-test feedback enters from the second attempt onward). An expanded 20-instance atomic suite with additional regression traps ships in the repository (`datasets/openworld-swebench/`).

Composition, nesting, and changing rules (E30–E32). Three experiments exercise `CompositeWorld`, a wrapper whose state nests its children's states under namespace keys. Children run *unmodified*: the composite slices a child's namespace out, steps the child's own transition, and writes it back. Coupling is explicit—sideways through `Bridge`s (an ordinary transition over the

two-slot dict of the coupled children's states, so the same synthesis-and-verification pipeline applies to bridges unchanged), upward through `Aggregator`s (derived from leaves, never simulated), and agents travel between children over `Route`s whose crossing effects (tolls, vetoes) are themselves verifiable transitions. `PhasedTransition` handles rules that change over time: ordered (trigger, transition) phases with sequential, irreversible advance, every phase constructed and verified *before* the run. E30 holds one system fixed—16 internal rules plus four ring couplings—and varies only how synthesis is posed: one monolithic prompt over a flat namespaced state versus eight small prompts (four 4-rule sectors, four one-rule bridges), each verified independently and assembled into a `CompositeWorld`; both conditions score on the same ground-truth probe suite (7B generator, three replicates each). E31 replays a 20-step script (leaf actions, ticks, and two travel attempts across a toll route) on a three-level composite against an independent flat-dict oracle that imports nothing from the composition machinery, checking leaf exactness, aggregator consistency, money conservation, and agent-registry agreement at every step. E32 declares a market whose policy changes at a fixed step mid-rollout and compares phased synthesis (each phase from its own rule text alone), monolithic synthesis from the combined rules-with-change text, and the LLM next-state proxy, on closed-loop rollouts crossing the boundary.

Models and hardware. All experiments use local models served by Ollama on an Apple-silicon laptop: `qwen2.5:7b` (generator, judge, critic in E3), `qwen2.5:3b` (small generator in E2/E3), `qwen2.5:1.5b` (repair agent in E5/E6/E13/E17), and—for cross-family replication (E16)—`llama3.1:8b` (Meta) and `gemma2:9b` (Google). The trained baselines in E12/E19 are plain numpy models. No cloud APIs, no fine-tuning, and no training compute beyond the baselines' seconds-scale fitting.

Baselines. The learned-dynamics family is represented two ways. (i) `LLMTransition`: the same backbone predicting each next state directly from the same description and rules—sharing the symbolic engine's knowledge while exhibiting the per-step stochastic prediction structure of learned models. (ii) *Genuinely trained* models (E12): a two-hidden-layer MLP and a 1-nearest-neighbor memorizer, each trained on $K \in \{100, 1000, 10000\}$ environment transitions collected by a random policy—real learned dynamics with a measured sample-efficiency curve. We are explicit that neither is a trained Dreamer; laptop-scale reimplementation of the pixel-native family is infeasible, which is itself part of the comparison (DreamerV3-class systems require millions of environment steps [8]).

Metrics and statistics. Rate metrics carry 95% Wilson confidence intervals. Dynamics fidelity uses exact-state match (every field equal), first-divergence step, and final-state L^1 error. Program repair reports pass@1 and pass@budget (4 attempts); the controlled selection comparison (E13) is a paired design on shared candidate sets with an exact two-sided McNemar test. Judge alignment uses Spearman rank correlation with permutation p-values (10,000 shuffles). Seeds are fixed in each script; every number in this paper regenerates via `python3 scripts/make_paper_assets.py`.

Calibration, pilots, and internal review. Design changes made after pilots are reported rather than hidden: (a) the verifier ablation (E3) uses the 3B generator at high temperature because the 7B generator's first attempts passed every gate, leaving nothing for conditions to discriminate; (b) the repair agent in E5/E6/E13 is the 1.5B model because both the 7B and 3B agents saturated the original benchmark (pass@1 = 100%)—with failing-test feedback in the state, these archetypes are easy for capable models. The capability ladder (1.5B: 70%, 3B and 7B: 100% on the original suite) is itself a finding. Experiments E11–E15, the expanded 20-task benchmark, and the statistical tests were added in a first revision in response to an internal adversarial review (both review rounds ship

in the repository at `docs/review/`), which demanded a trained baseline, multi-world replication, paired judge controls, and significance testing; a second review round demanded cross-family replication (E16), a powered judge comparison (E17), a filter-vs-repair decomposition (E18), and a scale ladder with a stronger learned baseline (E19); a third round demanded a complexity stress test (E20), stochastic-dynamics support and verification (E21), a planning-utility demonstration (E22), and an oracle-free error signal (E23); a fourth round, written from a moral philosopher's standpoint, demanded that the dial system's consequentialist commitments be made explicit and structurally broadened (E24–E27, §5.7). Experiment numbering preserves provenance; E14 was absorbed into the benchmark expansion and the number is retired. All LLM-dependent numbers are specific to the stated Ollama snapshots and Q4_K_M quantization; absolute rates may shift under other runtimes, though the structural comparisons should not.

5 Results

5.1 Head-to-head: symbolic vs. learned-style dynamics

Table 2 and Figure 1 present the primary comparison (E1, E4, E10), and Table 3 replicates the protocol across all three instrumented worlds with eight action scripts each (E11). The verified synthesized engines are exact on 24/24 rollouts (95% CI [0.86, 1.00]) against 0/24 (CI [0.00, 0.14]) for per-step LLM prediction, and—because planning executes Python rather than a forward pass—roll out at 14,997 steps/s against the LLM engine's 0.32 steps/s, after a one-time synthesis cost of a few seconds.

Engine	First divergence (step)	Final L1 error	OOD exact (10×)	Steps/s
Symbolic (verified code, ours)	21.0	0.0	100%	14,997
LLM next-state (proxy)	2.3	10.3	40%	0

Table 2: Head-to-head engine comparison on the sprint world (E1, E4, E10): mean first-divergence step and final L^1 error over three 20-step rollouts against a ground-truth oracle; exact transition accuracy on ten 10× out-of-distribution probes; and rollout throughput. A first divergence of 21 means no divergence occurred within the rollout.

World	Code: exact rollouts	LLM: exact rollouts	LLM first divergence
orchard	8/8	0/8	11.2
sprint	8/8	0/8	1.4
triage	8/8	0/8	2.0
Total	**24/24** [0.86, 1.00]	0/24 [0.00, 0.14]	—

Table 3: Multi-world replication (E11): exact 20-step rollouts per world, eight scripts each, dynamics synthesized once per world by the 7B generator. Totals carry 95% Wilson CIs.

5.2 The central result

Verified code synthesis eliminates compounding rollout error at zero training cost. The mechanism is structural, not statistical. A learned (or per-step predicted) transition is sampled each step, so per-step error ϵ compounds roughly as $1-(1-\epsilon)^t$: with the measured single-transition error of the LLM engine (60% of probes wrong), divergence by step 2.3 is exactly what the arithmetic

predicts, and twenty-step plans are fiction (final L^1 error 10.3). The symbolic engine pays its entire error budget *once*, at synthesis time, where it is auditable: if the accepted program is right, every rollout of any depth is right, bit-exactly. This is the CWM hypothesis [23, 9] made measurable on consumer hardware.

5.3 Against genuinely trained dynamics

The LLM next-state engine is a structural proxy; E12 adds real learned baselines trained on environment transitions (Figure 3). An MLP and a 1-NN memorizer were each trained on $K \in \{100, 1000, 10000\}$ random-policy transitions and held to the same evaluation as the synthesized program. Two results stand out. First, *sample efficiency*: even at $K = 10{,}000$ the MLP reaches only 50% exact accuracy on the branch-covering probe suite and 0% at 10× scale; the synthesized program scores 100% on both from *zero* transitions—its input is the rule text. Second, a measurement caution: at $K = 10{,}000$ the 1-NN memorizer completes 8/8 on-policy rollouts exactly while scoring only 30% on the probe suite—on-policy rollout fidelity can be *memorized* without learning the rules, which is precisely why branch-covering probes and OOD evaluation, not rollout agreement alone, are the right instruments for dynamics quality.

E19 extends both axes (Table 12). A *stronger* learned baseline—delta-state target, double capacity, lr-decayed training—does not improve on the original MLP (50% in distribution) and collapses identically out of distribution (0% at both 10× and 100×): the failure is not under-tuning but extrapolation itself. Across a 1×/10×/100× scale ladder the synthesized program is exact everywhere (100% at 100×); the LLM next-state engine is roughly scale-*insensitive* but uniformly unreliable (∼40–50% at every scale)—it does not degrade OOD because it was never anchored to the distribution in the first place.

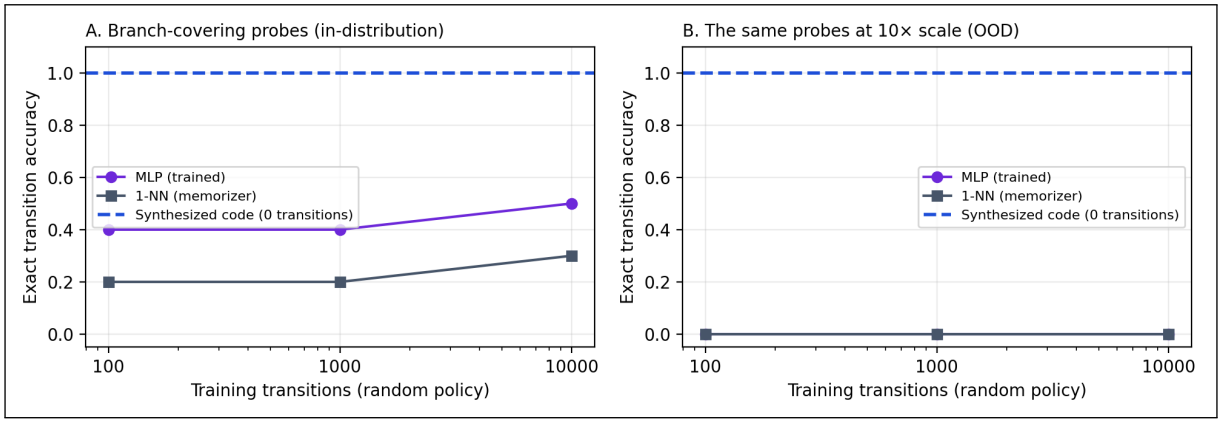

Figure 3: **Trained baselines vs. the zero-transition program (E12).** Exact transition accuracy of an MLP and a 1-NN memorizer trained on K random-policy transitions of the sprint world, on branch-covering probes in-distribution (A) and at 10× scale (B). The dashed line is the synthesized program, which uses no transitions at all. No trained baseline transfers out of distribution.

5.3.1 Induction on equal footing (E37)

A fair objection to the comparison so far is an *information asymmetry*: the synthesizer is handed the rules in natural language while the learned baselines must infer dynamics from sampled transitions. E37 removes it (Table 4). The LLM is given *only* observed $(\text{state}, \text{action}, \text{next})$ transitions—no rule text—and must induce a `transition()` program, accepted only if it reproduces the observed

transitions; it is then held to the same in-dist and 10× OOD probe suites as the MLP and 1-NN trained on the identical data. Two findings result, both honest. First, on equal information code induction beats statistical learning out of distribution by a wide margin: 0.43 versus 0.00 (MLP) and 0.00 (1-NN), and the advantage is *structural*—induced-code accuracy is identical in-distribution and at 10× scale (0.43 vs 0.43, on every replicate), because the model induces symbolic rules that extrapolate, whereas the learned baselines fit magnitudes and collapse to zero OOD on every replicate. The compounding/extrapolation advantage is therefore not an artifact of being handed the rules. Second, induction is *imperfect*: the 7B model recovers only 0.43 of the probe behavior from traces (it misreads the subtle `debt`-dependent interaction), well below the rule-text anchor's 1.00. The gap between 1.00 (rules) and 0.43 (traces) is what the specification genuinely contributes—neither nothing nor everything—and is the honest measure of the asymmetry the headline comparison carried. Larger budgets do not close it here: the reachable transition set under a random policy saturates, so K=100 and K=1000 give the inducer essentially the same distinct examples.

Method	Information given	In-dist.	10× OOD
Code synthesis (rules)	rule text, 0 transitions	1.00	1.00
Equal information: 1000 random-policy transitions, no rule text			
Code induction	traces only	0.43	0.43
MLP	traces only	0.40	0.00
1-NN memorizer	traces only	0.17	0.00

Table 4: E37, equal-information induction: exact probe accuracy when the LLM must induce dynamics from transitions alone (no rule text), against the MLP and 1-NN on the same data, plus the rule-text synthesis anchor. Code induction is imperfect but scale-invariant (in-dist = OOD); the learned baselines collapse out of distribution; the rules-vs-traces gap measures what the specification buys.

5.4 Supporting ablations

5.4.1 What each verification gate buys

E3 ablates the acceptance regime over eight paired generations per condition, using the 3B generator at high temperature so that defective candidates actually occur (Figure 4). Blind acceptance of the first code block yields mean ground-truth probe accuracy 0.46; syntax-only checking adds nothing (0.46—this generator's failures parse fine); the full behavioral gate (sandboxed smoke-runs + invariants + repair loop) lifts accuracy to 0.65, and adding the 7B semantic critic reaches 0.70—a +0.24 absolute gain over blind acceptance. Notably, no 3B generation achieved bit-exact dynamics under any regime (the 7B generator routinely does, §5.4.2): verification buys a large, measurable correctness margin, but cannot substitute for generator capability. The residual gap is semantic—code that runs and respects invariants can still misread a rule—which is the gap the probe-and-tighten workflow (§6) targets.

5.4.2 Synthesis reliability across model scale and family

E2 attempts compilation five times per world per model; E16 replicates the protocol with generators from two other vendors (Table 10, Appendix A). Every backbone reaches 100% verified acceptance within four iterations, but semantic quality varies: accepted 7B Qwen code matches ground-truth probes at 0.98 versus 0.87 for the 3B generator, while Llama-3.1-8B reaches 0.85 and Gemma-2-9B is *rule-perfect on every attempt* (1.00), at mean synthesis costs of 9 s and 4 s per world. The

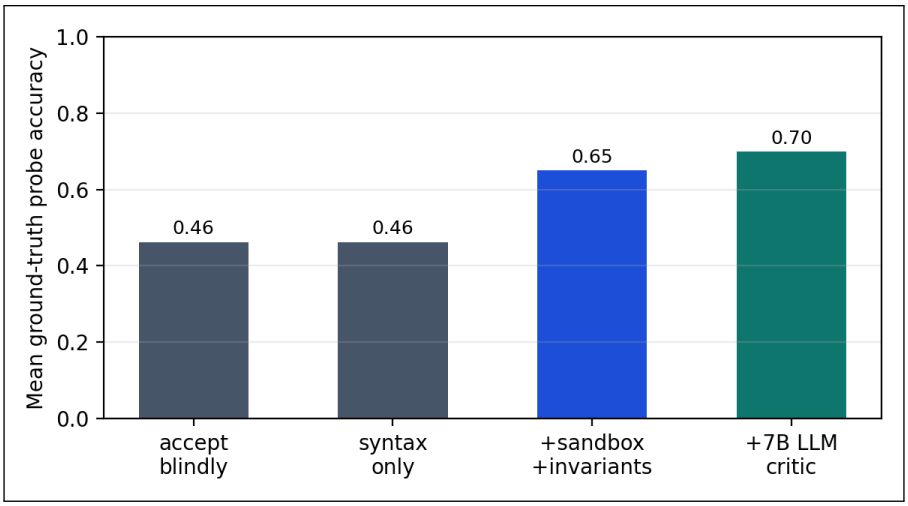

Figure 4: **Verifier ablation (E3).** Semantic correctness of accepted sprint-world dynamics under four acceptance regimes; eight paired high-temperature 3B generations per condition, graded against ground truth on a branch-covering probe suite.

paradigm is not a Qwen artifact: three model families synthesize verified dynamics in seconds on one laptop. Verification guarantees executable, invariant-respecting code—not rule-perfect code; generator capability still buys semantic fidelity.

E18 decomposes *what verification does* (Table 11): with the same full gate, accepting only first-shot candidates (no repair loop) accepts 62% of runs at probe accuracy 0.62 among accepted, while enabling the feedback loop lifts acceptance to 100% at 0.65. The filter is what protects quality; the repair loop is what recovers throughput—it converts rejected generators into accepted ones without degrading what gets through.

5.5 Agents-as-a-judge

E13/E17 form the controlled comparison (Figure 5, Table 5): for each paired round, the 1.5B proposer samples three candidate patches *once*, and four selection strategies act on the same set. E13's 60 rounds put the judge at 88% versus 82% for random selection (McNemar $p = 0.289$); the round-2 review asked for power, so E17 tripled the proposer seeds. Pooled over 120 rounds (20 tasks $\times$ 6 seeds): first candidate 78%, random 78%, judge 85%, oracle ceiling 91%. The judge-vs-random margin replicated in direction and sharpened—15 versus 6 discordant rounds, exact McNemar $p = 0.078$—but remains short of the conventional threshold; the judge recovers roughly half of the random-to-oracle gap, and we report the effect as estimated, not established. The position-bias audit is the sharper finding: asked again with candidates in reversed order, the judge's raw choices match on only 33% of pooled rounds—yet its *accuracy* is order-stable, picking a passing candidate 84% of the time forward and 86% reversed on discriminative rounds. Most inconsistency falls on rounds where several candidates pass and the choice is inconsequential; the bias is real, but it costs correctness little here. Cost accounting: judge-of-3 spends four model calls per round (three 1.5B generations plus one 7B judgment) against one for the single-proposal baseline, so the 85%-vs-78% margin is bought at roughly 4$\times$ inference cost—worth it when test execution is expensive or attempts are budgeted, not when proposals are free.

In the multi-attempt protocol (E5/E6), a single low-temperature 1.5B proposal solves 80% of tasks first-try, and feedback-driven retries recover nothing (pass@4 80%): the small agent resubmits near-identical wrong patches, a local minimum. Judge-selected high-temperature candidates trade

first-attempt accuracy (70% pass@1) for budgeted recovery (85% pass@4)—diverse sampling plus judgment escapes minima that deterministic retries cannot, at the cost of noisier first shots. E7 and E15 close the loop on judge-as-objective: across twelve dial-swept triage episodes, judge rubric scores track the ground-truth aggregate at Spearman $\rho = 0.75$ (permutation $p = 0.0057$); a paraphrased rubric is directionally consistent but weaker and only marginal ($\rho = 0.58$, $p = 0.0657$)—rubric wording matters. Together: judges are useful selectors and plausible objectives, but their verdicts should gate, not replace, programmatic verification.

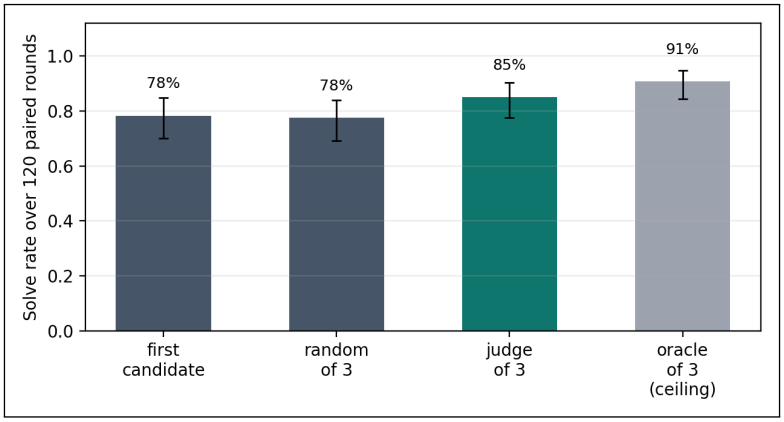

Figure 5: **Controlled judge-selection comparison (E13+E17, pooled).** Four selection strategies over the same three 1.5B candidate patches per round, 120 paired rounds; whiskers are 95% Wilson intervals. Random-of-3 isolates the diversity effect; the judge's margin over it estimates the value of judgment (McNemar $p = 0.078$, marginal).

Selection strategy	Solves	Rate (95% CI)
First candidate (no selection)	49/60	82% [0.70, 0.89]
Random of 3 (diversity only)	49/60	82% [0.70, 0.89]
Judge of 3 (ours)	53/60	88% [0.78, 0.94]
Oracle of 3 (ceiling)	56/60	93% [0.84, 0.97]

Table 5: E13 selection strategies on shared candidate sets (60 rounds: 20 tasks × 3 proposer seeds).

5.6 Steerability: the moral dial

E8 sweeps the moral weight $\lambda \in [0, 1]$ on the commons world (Figure 6). All 11 of 11 sweep points are Pareto-optimal, fairness is monotone in λ, and the Nash bargaining point sits strictly interior at $\lambda = 0.8$—the "interior optimum" behavior predicted by the tunable-morality literature, reproduced here with a dial that can be turned mid-simulation rather than a retrained model [21, 11, 22].

5.7 The structure of the moral filter

A fourth review round, written from a moral philosopher's standpoint (`docs/review/`), pressed a charge the engineering rounds could not: the dial system is not a neutral container for ethics. Everything expressible as `Objective+Dial` reduces to $\sum_i w_i o_i$—act consequentialism with a linear social welfare function—so the "morality dial" selects points in one theory's parameter space, not among theories. Four experiments, enabled by a new `openworld.ethics` module (Rawlsian and

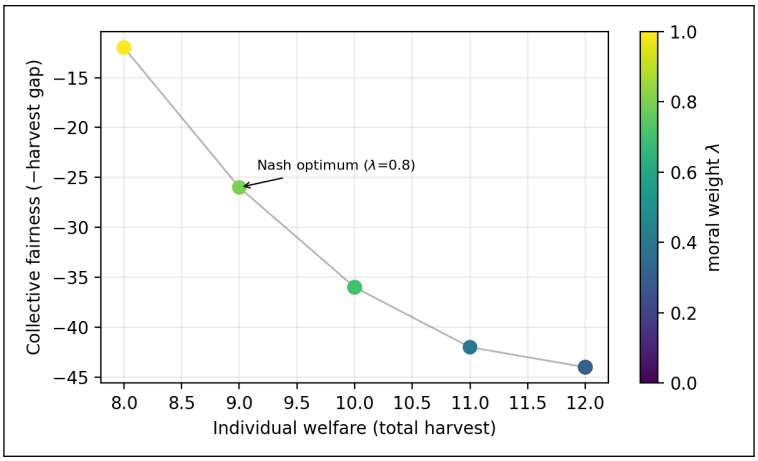

Figure 6: **Tunable morality (E8).** The welfare–fairness frontier traced by sweeping the moral dial λ on the commons world. Every point is Pareto-optimal; the Nash bargaining optimum is interior. Values are declared, weighted artifacts—moving along this frontier requires no retraining.

lexicographic aggregators, deontological constraints as action vetoes, and a moral parliament), measure what that restriction costs and what lifting it buys.

Aggregation is an ethical commitment (E24). On a scarce commons where one agent harvests faster than the other, the utilitarian sum, Rawlsian maximin [18], and leximin choose different operating points from the same outcome table: the sum is *indifferent* between harvests of $(6, 3)$ and $(5, 4)$—both total 9—and so picks the unequal allocation arbitrarily, while maximin and leximin select the restraint level that lifts the worst-off from 3 to 4 at zero total cost. Distribution-blindness is not a hypothetical defect of the weighted sum; it selects a different world even when an egalitarian free lunch exists.

Side constraints cannot be simulated by weights (E25). A myopic cost-weighted nurse becomes *impermissible* at $\lambda > 1$: the weighting makes abandoning waiting critical patients optimal (24 violations across the sweep, four deteriorations, outcomes collapsing to -8). A Nozickian side constraint [16] —"never choose another action while a critical patient waits," enforced as a veto on the action set, not a penalty in the score—eliminates every violation at every λ, and here costs nothing in the dial's permissible region (0 outcome points) while *rescuing* the impermissible one. Penalties are just more utilitarianism; vetoes are a different kind of object, and the framework now has both.

Moral uncertainty and the parliament (E26). An operator who does not know the right theory should hedge rather than maximize within one theory [10]. Delegates for utilitarian, Rawlsian, and deontological positions vote (credence-weighted Borda) over each action; on the triage shift, the parliament is never the strictly worst agent under *any* theory's own lens—and notably it rescues the lone Rawlsian delegate from itself: pure one-step maximin lets critical patients deteriorate (worst-off welfare -6, two duty violations), while the parliament's mixed counsel reaches the same outcome as the best single-theory agents (-3, zero violations).

Pluralism is visible in the judge (E27). Grading the same twelve dial-swept trajectories under utilitarian, deontological, and care-ethics rubrics yields orderings that correlate only partially

(pairwise Spearman 0.62–0.93; the deontological and care rubrics track the dial differently than the utilitarian one). Rubric framing is a value choice, not a wording choice—a quantitative counterpart to Berlin's incommensurability claim [3] and a caution for anyone deploying a single "ethics rubric" judge.

5.8 Configuration search

E9 compares tuning strategies on the triage auto-configuration problem (two moral dials × protocol × budget) over ten seeds at a 200-trial budget. All strategies reach the optimal plateau on all seeds; they differ in how quickly a first *solving* configuration is found (Table 13, Appendix A). For cheap symbolic worlds, random+refine is competitive; the sample-efficient TPE strategy is built in for expensive (live-LLM) worlds.

5.9 From fidelity to decisions: planning

World models exist to improve decisions; E22 closes that chain (Table 6). A model-predictive planner runs exhaustive depth-3 lookahead inside its world model and executes the best action in the ground-truth environment. Planning through the verified synthesized program scores 7.5—*identical* to planning through the ground truth itself (7.5), as bit-exactness guarantees—against 5.0 for a reactive heuristic and 3.8 for random actions, with the whole 12-step episode planned in 0.03 s. Planning through the LLM next-state engine is confined to depth 2 by cost (86 s per episode) and scores 0.0—*worse than no model at all*: lookahead through hallucinated transitions steered the agent away from productive actions entirely. Fidelity and throughput are not abstract virtues; they are the difference between planning that helps and planning that harms.

Policy	Episode score	Planning time (s)
Lookahead d=3 via synthesized code	7.5	0.03
Lookahead d=3 via ground truth (bound)	7.5	0.00
Lookahead d=2 via LLM next-state	0.0	85.85
Reactive heuristic (no model)	5.0	0.00
Random policy (5 seeds, mean)	3.8	—

Table 6: E22: model-predictive lookahead on the sprint world, plans executed in the ground-truth environment. Value function: shipped − bugs − 0.5·debt after 12 steps.

5.10 Scope boundaries and oracle-free verification

Three round-3 experiments chart where the paradigm holds and what to do at the edge.

The complexity cliff (E20). Parametric worlds with R interacting rules (conditions reading pre-action state, summed effects, clamping) were synthesized from rule text at $R \in \{4, 8, 12, 16\}$ (Figure 7). Mean probe accuracy falls from 1.00 at R=4 to 0.53 at R=8, 0.25 at R=12, and 0.15 at R=16: monolithic single-function synthesis with a 7B generator degrades sharply once roughly eight coupled rules must be implemented at once. This is the paradigm's present boundary at this model scale, and it motivates compositional synthesis—one expert per rule, as in PoE-World [17]—as the structural fix.

Stochastic dynamics (E21). Threading a seed through the state (`rng_seed` in, derived seed out) extends verified synthesis to stochastic worlds without sacrificing replayability. On a queueing world with a declared 0.3 arrival probability, accepted programs (67% of attempts) matched the declared distribution within 1.2 percentage points over 2,000 seeded transitions, kept the deterministic bookkeeping perfect (100%), replayed bit-exactly, and in fact matched the hand-written stochastic oracle bit-for-bit (100%).

Oracle-free error detection (E23). Practitioners lack the hand-written oracles our evaluations use; they can, however, synthesize the dynamics several times. Across 30 stored programs and 900 program–probe pairs, disagreement with the ensemble majority detected ground-truth errors with 100% precision and 100% recall—the majority vote reconstructed the oracle exactly on these worlds—and a program's agreement rate ranked its true accuracy perfectly (Spearman $\rho = 1.00$). The caveat is structural: majority-as-oracle fails under correlated errors (e.g., every model misreading the same ambiguous rule), so ensemble disagreement is a cheap screen, not a proof; it composes naturally with the invariant and critic gates.

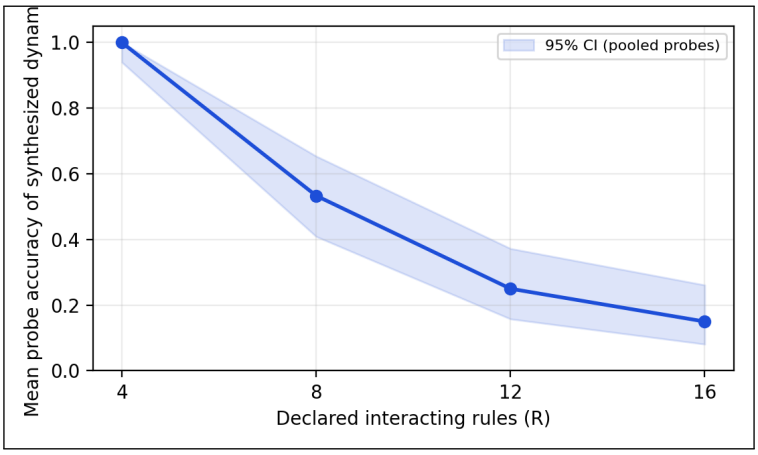

Figure 7: **The complexity cliff (E20).** Probe accuracy of 7B-synthesized dynamics versus the number of declared interacting rules; three syntheses per point, shaded band is the 95% CI over pooled probes. Monolithic synthesis is reliable at R=4 and unreliable past R≈8.

5.11 End-to-end evaluation

The coding world is itself the end-to-end test: world declaration, bit-exact test-execution dynamics, an LLM agent in the loop, feedback-driven repair, and judge-based selection compose into a system that repairs 85% of benchmark defects with 1.5B/7B local models and no training (and 100% with a 7B agent alone)—every component of the framework exercised against a verifiable external standard.

5.12 When does the feedback loop pay off? (E28–E29)

The in-world loop — exact failing-test feedback flowing from the world's dynamics into the next attempt — is the framework's central mechanism for repair, and E28–E29 ask directly what it is worth (Table 7). The design isolates the loop: in-world attempt 1 uses the *same* prompt as single-shot, so the two conditions can differ only through feedback-driven iteration.

On the atomic suite, single-shot pass@1 scales cleanly with model size (0.17 → 0.50 → 1.00), so the benchmark discriminates capability — but the loop adds nothing at any rung ($\Delta \approx 0$; the 7B entry's small negative value is sampling noise under temperature at $n = 6$, since its single-shot already solves every instance). An atomic, issue-described bug is one-shot fixable or it is not; feedback has nothing left to contribute. The staged suite changes the structure, not the harness: the issue-visible fix surfaces a latent second failing test that no prompt describes. There the loop becomes load-bearing, and its value is *capability-gated*: the 1.5B model stays flat (+0.00 — it cannot convert the feedback it is shown), the 3B model gains $\Delta = +0.13$, and the 7B model gains $\Delta = +0.33$ (single-shot 0.13 → 0.47 at budget 4). Two conclusions follow. First, the world model's feedback channel pays only when the base model is strong enough to act on structured feedback, and pays more with scale. Second, whether the channel is measurable at all is a property of task *structure*: multi-stage defects — the shape real SWE-bench issues take — expose it; single-edit defects mask it. The caveats are the usual ones at this scale: n is small and the Wilson intervals are wide (per-instance paired records ship with the results for exact tests), and absolute rates are specific to the stated local quantized snapshots.

Suite	Model	SS pass@1	IW pass@1	IW pass@4	Δ	Mean att.
Atomic ($n = 6$)	`qwen2.5:1.5b`	0.17	0.17	0.17	+0.00	3.5
	`qwen2.5:3b`	0.50	0.50	0.50	+0.00	2.5
	`qwen2.5:7b`	1.00	0.67	0.83	−0.17	1.7
Staged ($n = 15$)	`qwen2.5:1.5b`	0.07	0.07	0.07	+0.00	3.8
	`qwen2.5:3b`	0.13	0.13	0.27	+0.13	3.3
	`qwen2.5:7b`	0.13	0.13	0.47	+0.33	3.0

Table 7: E28–E29: paired single-shot (SS) vs. in-world (IW) repair on the atomic and staged suites. Δ is in-world pass@4 − single-shot pass@1; in-world attempt 1 shares the single-shot prompt, so Δ measures feedback-driven iteration alone.

5.13 Composition defeats the complexity cliff (E30–E32)

Figure 8 shows the anatomy under test: a composite world is itself a world, its children run unmodified, and every coupling channel is an explicit, independently verifiable object—bridges sideways, derived aggregators upward, bindings downward, routes for agents.

E20 (§5.10, Figure 7) left monolithic synthesis collapsing past $R{\approx}8$ coupled rules and named composition as the structural fix; E30 delivers it (Figure 9, Table 8). The *same* 16-rule system that 7B monolithic synthesis renders at 0.31 probe accuracy (95% CI [0.21, 0.42]) reaches 0.92 (CI [0.83, 0.96]) when posed as four 4-rule children plus verified bridges—every synthesized piece below the E20 cliff. Acceptance is identical in the two conditions (every attempt passed the gate), so the verifier is not the difference; the decomposition is. This is the one-expert-per-piece structure PoE-World argues for [17], with each piece here passing the same verification pipeline as a whole world.

The composition machinery itself adds no error. In E31, a three-level composite (region → country → city) with a goods bridge, chained aggregators, and a toll route is replayed against an independent flat-dict oracle sharing no code with the framework: all 20 steps are exact (20/20 on leaf exactness, aggregate consistency, money conservation, and agent-registry agreement, including one paid and one vetoed crossing). This holds essentially by construction—children run unmodified, aggregates are derived rather than simulated—but the independent oracle makes it evidence rather than assertion. The same hierarchy also defines what an agent embedded in it can see (Figure 10):

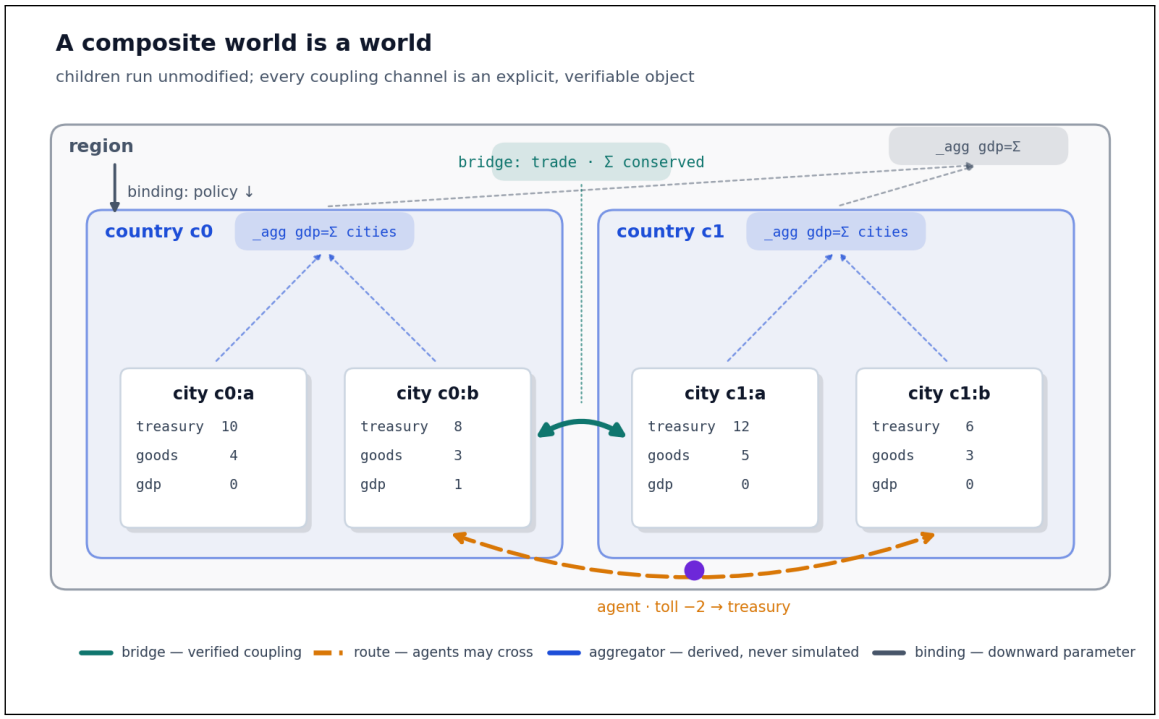

Figure 8: **Anatomy of a composite world** (the E31 system, real state values). Two country worlds, each containing two unmodified city worlds, compose into a region. The four coupling channels are explicit objects: a verified *bridge* carries trade under a conservation invariant; a dashed *route* lets an agent cross (paying a toll into the destination treasury); dashed upward arrows are *aggregators*—parent summaries derived from leaves, never independently simulated; the solid downward arrow is a *binding* injecting a parent parameter into a child.

full detail at its own node, derived aggregates of every ancestor, summaries of route-adjacent neighbors, and nothing else—scoped observation that lets planners reason locally against exact dynamics without ever holding the macrostate.

The three mechanisms interact, and the interaction is where realism lives: rules change *inside* a composition, and embodied agents respond by moving. E33 (Figure 11) is a deterministic demonstration of that loop. Two economies share a toll route; a greedy policy agent works wherever its scoped view shows the best rate. When the home economy's `PhasedTransition` enters austerity at step 9 (work yields zero and the *observed* rate collapses), the agent's next observation prices the change, and at step 10 it pays the 2-coin toll and re-locates. World GDP ends at 44 with the route against 24 without it—a mobility gain of 20. Composition provides somewhere to go, dynamic rules provide the reason, and traversal is how behavior adapts; remove any one channel (the stranded counterfactual removes the route) and the regime shock becomes a permanent flatline instead of a re-allocation.

E32 deserves an honest reading. Both phased and monolithic synthesized code track the regime switch exactly (3/3 and 3/3 exact full rollouts; pre-boundary, boundary, and post-boundary accuracy all perfect): a single declared rule change does not by itself defeat a 7B generator at this scale. The LLM next-state proxy collapses on the same script (0.20 pre-boundary, 0.00 after the switch). `PhasedTransition`'s value is therefore structural rather than a fidelity rescue: each phase is synthesized and verified independently against its own rule text, so changing rules *compose* with the E30 result—adding a regime adds a phase, instead of multiplying the complexity of a single synthesis prompt.

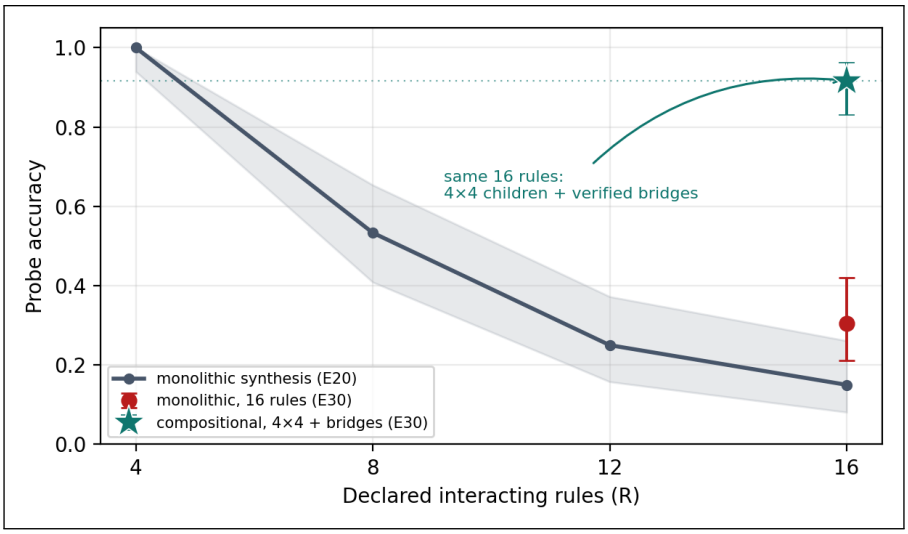

Figure 9: **Composition defeats the cliff.** The E20 curve (grey, with pooled 95% CI band): monolithic 7B synthesis accuracy falls as declared interacting rules grow. At R=16, E30's monolithic condition (red) lands on the fallen curve, while the same sixteen rules synthesized as four 4-rule children plus verified bridges (teal star) return to near-R=4 accuracy.

Finally, E34 puts the composite to work on the benchmark itself: a *sprint world* whose twenty children are the OpenWorld-SWE-bench repair worlds, unmodified, with aggregators exposing open tasks and failing-test totals, and an allocation policy deciding which task receives each of 80 repair attempts (model, prompts, and sampling identical to the standard protocol). The standard fixed-budget protocol solves 15/20—the first model-ladder numbers for the expanded 20-instance suite—while consuming only 36 attempts (Figure 12). Reallocating the stranded budget raises nothing: round-robin over open tasks also ends at 15/20, and a greedy closest-to-passing policy collapses to 1/20 by sinking 78 attempts into a single near-miss. Two mechanisms explain this, and both matter beyond this experiment. First, with the recipe's pinned sampling seed, a retry on an unchanged state is a verbatim replay—under pinned seeds, pass@k quietly degenerates toward pass@1, a property of the evaluation protocol worth stating explicitly. Second, the pathology is not the seed's: with per-attempt seeds restoring genuine retry diversity, round-robin still ends at 15/20 and greedy still tarpits at 1/20—committing to the nearest-to-done task is structurally wrong when that task is capability-bound, because it never gets done. The composite's measurable value here is efficiency and observability—round-robin reaches the plateau in roughly a third fewer attempts than the fixed protocol, and the sprint dashboard shows *where* attempts stop paying—not a higher ceiling. Allocation cannot buy what the model cannot produce.

5.14 The sprint allocation across a capability ladder (E35)

E34's finding—that reallocating a shared budget cannot raise the ceiling, and that a greedy "closest-to-passing" policy tarpits—was measured with a 1.5B/7B repair model. E35 re-runs the three allocation conditions across a capability ladder spanning three model families (Figure 13): qwen2.5:7b (the E34 anchor), deepseek-r1:14b, gpt-oss:20b, and qwen3-coder:30b. Two patterns emerge, both clean. First, the fixed-protocol ceiling rises monotonically with capability—15, 17, 19, 19 of 20—so the benchmark discriminates model strength as intended. Second, the greedy tarpit is *capability-gated*: catastrophic at 7B (1/20, it sinks the whole budget into one near-miss), it progressively defangs with scale and at gpt-oss:20b not only stops tarpitting but clears the *entire* board (20/20),

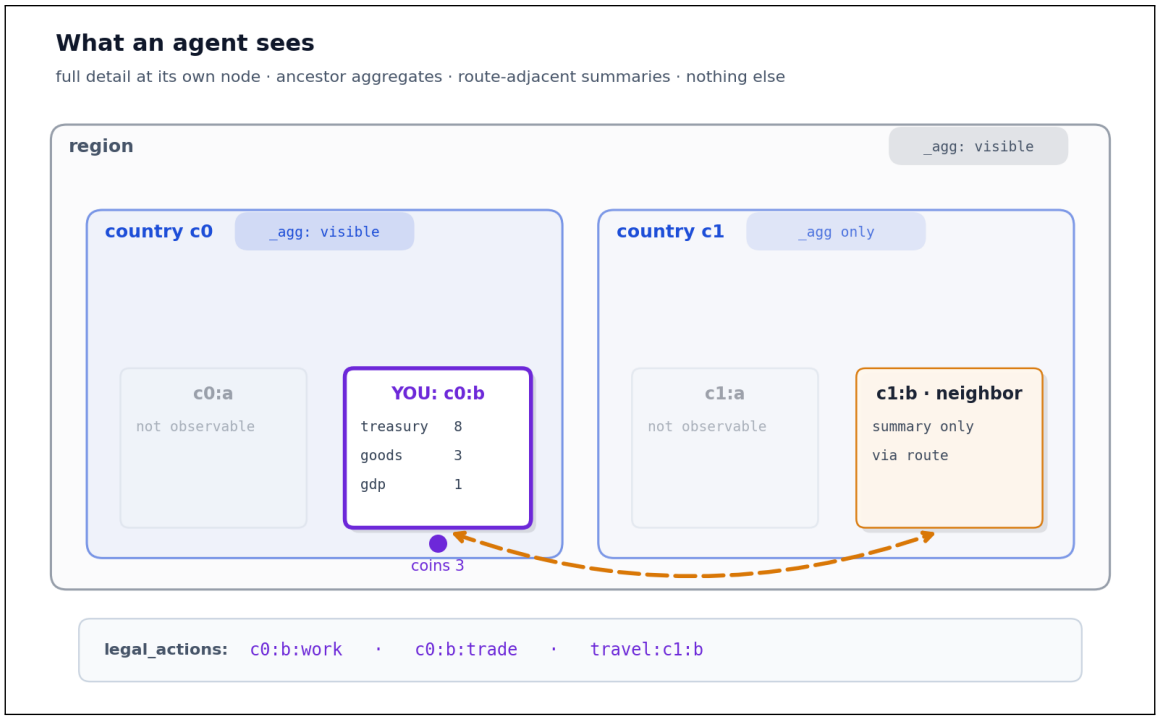

Figure 10: **What an agent sees** (E31, the trader at `c0:b` after step 1). Its own city is fully observable; ancestor levels expose only their derived `_agg` summaries; the route-adjacent city is a summary card; sibling cities are not observable at all. The `legal_actions` strip is the agent's full action menu: local actions plus one travel option per incident route.

E30 condition (same 16-rule system)	Accepted	Probe acc. (95% CI)
Monolithic (one 20-rule prompt, flat state)	100%	0.31 [0.21, 0.42]
Compositional (4 sectors × 4 rules + 4 bridges)	100%	0.92 [0.83, 0.96]

E32 engine (switch at step 10)	Pre	Boundary	Post	Exact rollouts
PhasedTransition (two verified phases)	1.00	1.00	1.00	3/3
Monolithic (combined rules-with-change text)	1.00	1.00	1.00	3/3
LLM next-state (proxy)	0.20	0.00	0.00	0/1

Table 8: E30 (top): the same 16-rule coupled system synthesized monolithically vs. compositionally (four 4-rule children plus four one-rule bridges); probe accuracy with pooled 95% CIs, three replicates per condition. E32 (bottom): per-step exact match against the oracle before, at, and after a declared regime switch, with exact full closed-loop rollouts; both synthesized conditions are perfect and the LLM next-state proxy collapses.

with qwen3-coder:30b at 19/20. The allocation pathology, in other words, is a symptom of a model that cannot finish the task it fixates on; give it enough capability and the fixation becomes harmless. Across all four models and three families the qualitative reading of E34 holds—allocation policy is dominated by base capability—now with the capability axis made explicit.

5.15 Composition yields better representations (E36)

E12 and E19 measured what a learner needs to match a *single* synthesized world; E36 asks the representational question directly on *factored* worlds, where composition's advantage should appear

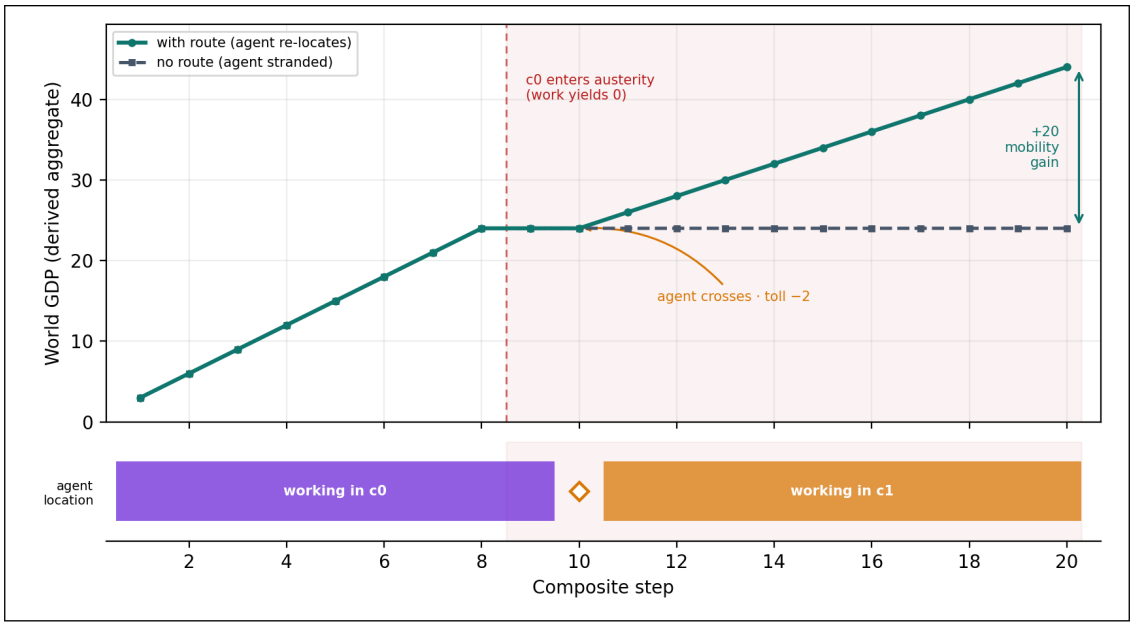

Figure 11: **When the rules change, agents move** (E33, deterministic). Top: world GDP with and without the route; the shaded band is the home economy's austerity regime, entered at step 9. The agent observes the collapsed rate, crosses at step 10 (toll −2), and growth resumes in the destination economy; the stranded counterfactual flatlines, leaving a +20 mobility gain. Bottom: the agent's location lane on the same time axis.

if it exists anywhere. The world is an economy of K parametric sectors (the E30 substrate); we sample transitions and learn, pitting a zoo of 9 monolithic learners on the joint state—linear/ridge regression, 1-NN and k-NN, kernel SVR, a Gaussian process, a random forest, histogram gradient boosting, a Koopman/EDMD operator on a degree-2 lift, and a two-hidden-layer MLP—against a composite of small per-sector learners and the exact symbolic composite, on three axes a learned dynamics model is judged by (Table 9, Figure 14). The comparison is capacity-fair: the monolith is given at least as many parameters as the composite's children combined (7203 vs. 6895 at K=5), so any gap is representational, not a matter of model size.

Compositional generalization. Training covers each sector's full marginal value range but only a thin slice of the joint product; the test set is the full product, dominated by sector-value *combinations* never seen together. The composite needs only the marginals—each child sees its whole small local space—so its accuracy is flat in K (learned-MLP 0.72–0.73; learned-trees and symbolic exact). The entire monolithic family needs the joint and collapses as the unseen fraction grows: every one of the 9 learners falls below 0.03 by K=5 *except* gradient boosting, the strongest, which still decays from 0.48 to 0.20—one to two orders of magnitude below the composite. Crucially, the collapse is not an MLP artifact: it holds across kernel methods, trees, Gaussian processes, and a Koopman operator alike, and factoring rescues a *non-MLP* child too—per-sector gradient boosting reaches 1.00 where joint gradient boosting cannot. *Interference.* Trained on the sectors in sequence, the monolith forgets the first the moment it learns the second (retained accuracy 0.03); the composite stores each child separately and retains 0.83. *Sample efficiency.* At 100 training transitions the composite reaches 0.66 against the monolith's 0.01, and the symbolic composite needs none. The decisive pattern is a three-tier gradient—monolithic learning collapses (every model class tried) → factored learning recovers most of it → symbolic factoring is exact and training-free. Factoring the representation, not scaling or swapping the learner, is what buys generalization here.

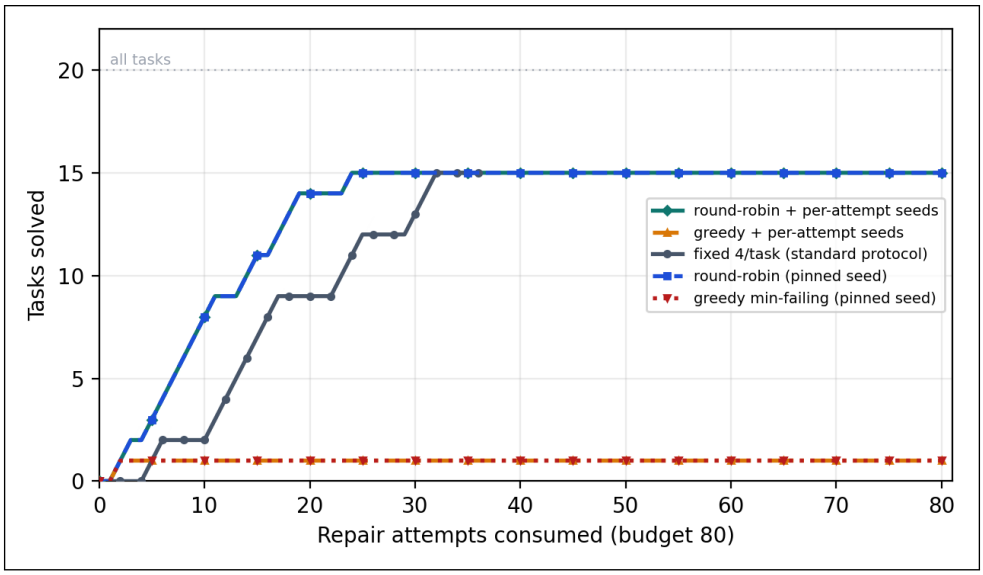

Figure 12: **The sprint world (E34):** tasks solved vs. repair attempts consumed on OpenWorld-SWE-bench (qwen2.5:7b, budget 80). The allocation policies act through the composite's global view. No policy beats the fixed protocol's ceiling—the unsolved tail is capability-bound, and greedy's closest-to-passing heuristic tarpits on a single task with or without retry diversity—but round-robin reaches the plateau markedly faster.

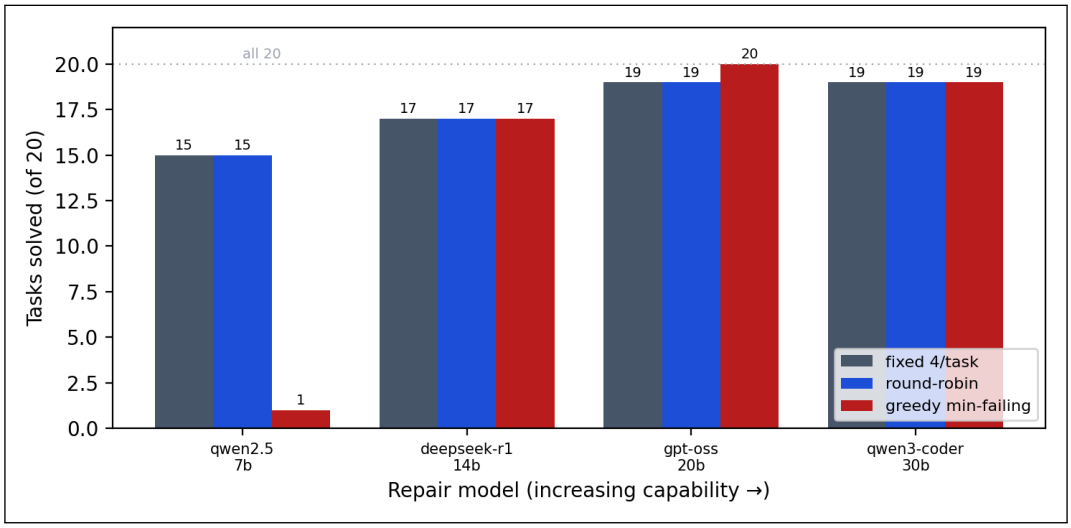

Figure 13: **The sprint allocation across a capability ladder (E35).** Tasks solved on OpenWorld-SWE-bench under the three allocation conditions, by repair model (increasing capability left to right; budget 80). The fixed-protocol ceiling rises with capability; the greedy tarpit (a lone solved task at 7B) defangs with scale, clearing all 20 at gpt-oss:20b.

5.16 Feeding the real world in: multimodal perception (E39–E40)

The worlds so far are driven by symbolic inputs. A *perception boundary* extends them to text, audio, and video without touching the verified core: an untrusted `Perceptor` maps raw input to a partial symbolic state update, a gate contract-checks it (owned fields, types, ranges), and only then does it reach the dynamics—before any transition runs. Raw media never enters state (only the resolved symbolic delta and a content hash), so determinism and bit-exact replay are preserved;

Representation	K=2	K=3	K=4	K=5
Monolithic learner over the joint state				
Linear / ridge	0.01	0.01	0.01	0.01
1-NN memorizer	0.02	0.03	0.01	0.01
k-NN (k=5)	0.03	0.03	0.01	0.01
Kernel SVR	0.02	0.01	0.00	0.01
Gaussian process	0.01	0.01	0.00	0.01
Random forest	0.02	0.02	0.01	0.01
Hist. gradient boosting	0.48	0.35	0.23	0.20
Koopman / EDMD (deg. 2)	0.02	0.02	0.01	0.01
MLP (2 hidden)	0.03	0.02	0.00	0.01
Composite of small worlds (ours)				
Composite-learned (MLP)	0.72	0.75	0.71	0.73
Composite-learned (trees)	1.00	1.00	1.00	1.00
Composite-symbolic	1.00	1.00	1.00	1.00

Table 9: E36 compositional generalization: exact next-state accuracy on *novel* sector combinations, by number of parts K. Every monolithic learner over the joint state collapses as K grows (gradient boosting, the strongest, decays from 0.48 to 0.20; the rest sit near chance); factoring the same problem into per-sector learners holds flat, and is exact with a strong child learner or with synthesized code.

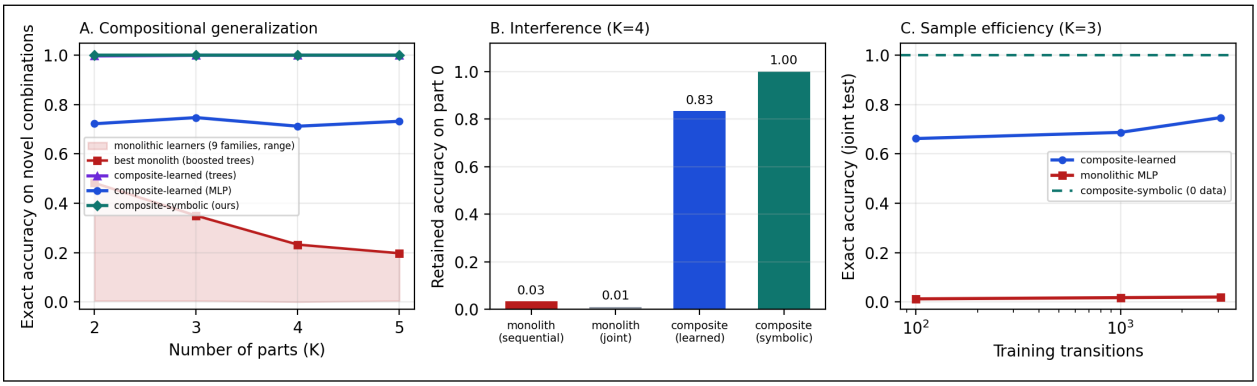

Figure 14: **Composition vs. monolithic learners (E36)** on a factored K-sector world, capacity-fair. **A:** exact accuracy on novel sector combinations—the whole monolithic family (shaded range over 9 learners; best, boosted trees, shown) collapses as K grows, while both composite learners and the symbolic composite stay flat near or at 1.0. **B:** accuracy retained on the first part after training through all K—the monolith suffers catastrophic interference, the composite does not. **C:** accuracy vs. training transitions—the composite learns from far less data, the symbolic composite from none.

perception is the one acknowledged fallible, non-deterministic layer, and is *measured*, never reported as verified.

This is what lets messy real-world observations drive an exact model of a real-world phenomenon. E40 takes a discrete SIR epidemic—the textbook case where the decision that matters is a multi-step *forecast*, not a one-step read—and drives it from perceived day-0 inputs (a text situation report and a ward-photo frame, resolved through the boundary into S, I, R), then rolls the verified dynamics forward 14 days. The perceived world model forecasts the day-14 infected count *exactly*, both in-distribution (1.00) and at 10× population scale (1.00), where an end-to-end MLP given the same inputs scores 0.01 in-distribution (0.05 within ±2) and 0.00 out of distribution (Figure 15A): learning a 14-step nonlinear outcome directly neither fits nor extrapolates, while rolling

a declared model forward is exact at any scale. Crucially, the value survives imperfect perception in a principled way. Because the dynamics add zero error, end-to-end forecast accuracy equals *perception* accuracy (Figure 15B): a perceptor right 75% of the time yields 75% correct forecasts, no worse. E39 confirms this decomposition holds exactly across perception error rates 0–0.9 (yes). The practical reading: a symbolic world model fed by perception models a real phenomenon as well as its declared rules and its perception allow—and the architecture keeps those two error sources separate and individually measurable, with the dynamics contributing none.

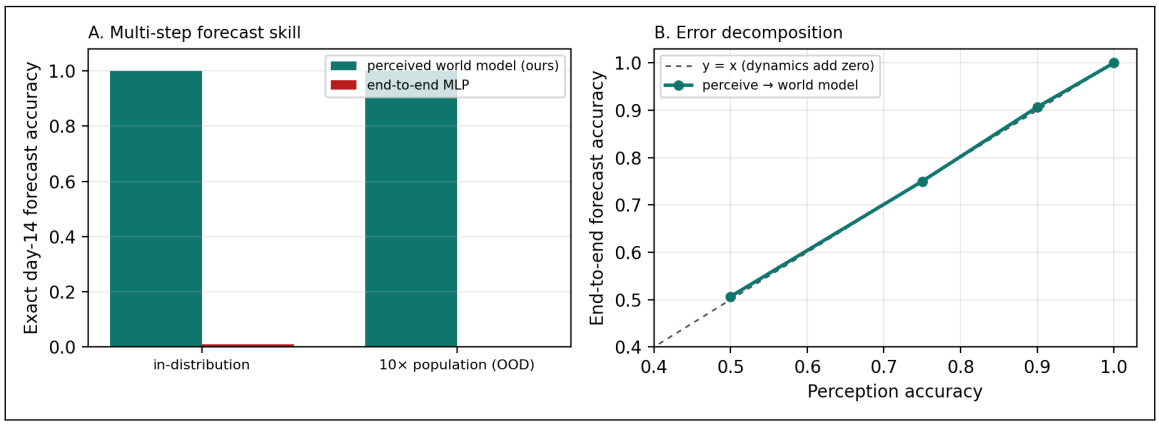

Figure 15: **Perceive-then-forecast (E40).** Multimodal day-0 inputs are perceived into the symbolic state of an SIR epidemic, then rolled forward 14 days by the verified world model. **A:** exact day-14 forecast accuracy—the perceived world model is exact in and out of distribution; the end-to-end MLP fails and collapses at 10× scale. **B:** end-to-end forecast accuracy equals perception accuracy (the dynamics add zero error), so the system degrades only as fast as its perception does.

5.17 Adapting to sudden, unannounced rule changes (E41)

E32 handled a regime switch whose timing was *known*, via a pre-verified `PhasedTransition`. E41 asks the harder question a non-stationary world poses: when the rules change suddenly and *without announcement*, who notices, and how fast do they recover? A reservoir tracks toward a hidden target at a hidden rate; that regime jumps at 2 unannounced times in a 120-step episode, and at each step the current level is read through the perception boundary and four predictors forecast the next level (Figure 16). A symbolic monitor that watches its own prediction error and re-identifies the rule from the post-change transient recovers in 1/1 step(s) at each change—snapping back to exact (0.94 overall). A standard sliding-window 1-NN recovers only as its window refills (12/22 steps; 0.66 overall, and only approximate), and a static model frozen on the first regime never recovers (0.26), trailed against an oracle that knows the change times (0-lag, exact). The cumulative-error curves (Figure 16A) make the difference plain: the symbolic monitor's regret is nearly flat with a small step at each change, while the frozen model's climbs without bound. Under noisy perception the ranking is unchanged and every method's error floor rises by exactly the perception error—the E39 decomposition, now riding on top of the adaptation dynamics. The mechanism behind the fast recovery is the same one the paper rests on: because the dynamics are an explicit, identifiable program, a handful of post-change observations re-pin the rule exactly, where a statistical model must re-estimate over a window. The boundary is equally explicit: this holds when the changed rule stays within the identifiable family the monitor searches; a change to an undeclarable rule admits no exact recovery.

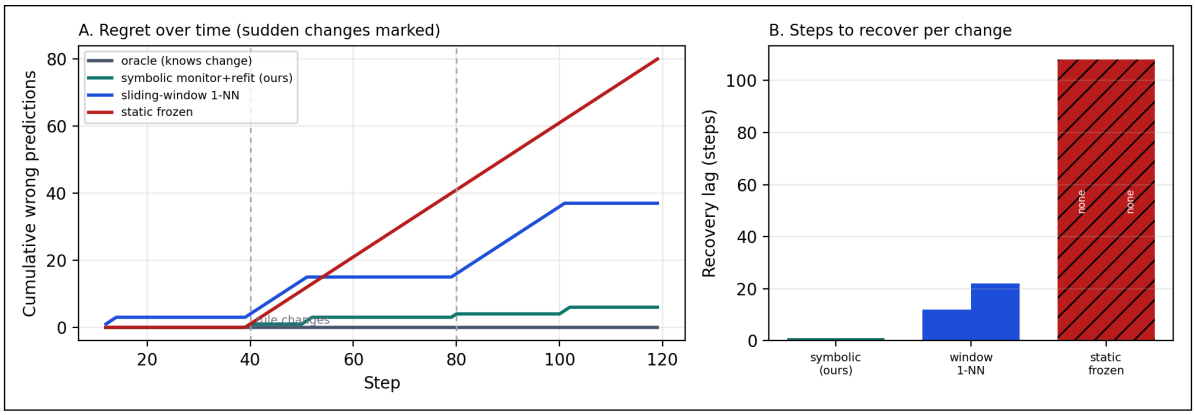

Figure 16: **Adapting to sudden unannounced rule changes (E41).** A reservoir's hidden regime jumps at the marked steps; predictions run through the perception boundary. **A:** cumulative wrong predictions—the symbolic monitor (ours) stays nearly flat, the sliding-window learner steps up and slowly recovers, the static-frozen model climbs without bound, the oracle is flat at zero. **B:** recovery lag per change—1/1 steps for the symbolic monitor (to exact) versus 12/22 for the window; the frozen model never recovers.

5.18 Agents traversing connected worlds with changing rules (E42)

E42 is the capstone: it combines composition, toll routes, non-stationary rules, and per-world perception, and asks whether an agent can track each world's rule *at the belief level* as it hops between them and the rules change—including changes that happen while it is away. Two reservoir worlds are joined by a toll `Route`; each has its own changing regime and its own perception boundary; an agent hops every 8 steps (19 arrivals, 10 tolls paid), and four belief models predict the current world's next state from the same perceived observations (Figure 17). The decisive contrast is between *separating* a belief per world and *sharing* one. The symbolic per-world monitor scores 0.94 and, crucially, returns to a world it left *unchanged* almost for free (recovery 0.2 steps), recovering to exact in 1 step when the world changed while it was away. The single shared learner scores 0.85—and pays a recovery cost of 1.1 steps even returning to a world that *did not change*, because learning the other world overwrote its belief: catastrophic interference (the E36 result) now induced purely by traversal. A per-world sliding window avoids interference (0.71) but does not reliably re-identify an away-change within a single visit. The lesson threads the whole paper together: separating dynamics into per-world programs—the composition principle—is what lets an agent carry one world's rule across an absence intact and re-pin it from a few observations when it has changed, where an entangled belief cannot hold two worlds at once. The boundary is the familiar one: exact re-identification holds while the changed rule stays in the identifiable family.

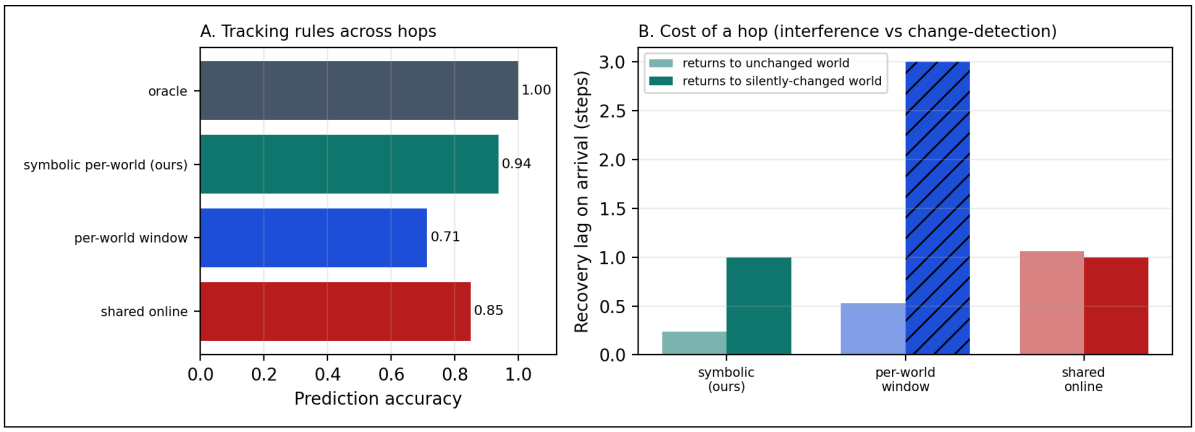

Figure 17: **Agents traversing connected worlds with changing rules (E42).** Two worlds joined by a toll route, each non-stationary, each with its own perception boundary; an agent hops between them. **A:** prediction accuracy tracking the current world's rule across hops. **B:** recovery lag on arrival, split by whether the world's rule changed while the agent was away. The shared-belief learner pays an interference cost even returning to an *unchanged* world; the per-world symbolic monitor returns nearly free and re-identifies away-changes in a step; the per-world window does not reliably recover from away-changes within a visit (hatched = no recovery).

6 Discussion and Limitations

Interpretation. The experiments quantify a structural trade. Learned world models buy pixel-native breadth with training compute and pay in compounding error, opacity, and frozen values. Verified code synthesis buys exactness, speed, OOD transfer, and auditability, and pays at the boundary of what can be declared symbolically. Within that boundary the results are one-sided: a laptop-scale local model, orchestrated as plan–generate–verify, produces simulators that are *perfect* where learned-style baselines are approximate. The practical workflow the ablations license: declare rules precisely; verify with invariants; probe the accepted artifact against intent; tighten and recompile. Each gate buys measurable correctness, and the artifact remains a diffable program at every stage.

Value alignment as an open specification. The dial experiments ground the alignment claim: because objectives are declared artifacts, the welfare–fairness frontier is *operational*—an operator moves along it in real time, and a tuner searches moral configurations jointly with world design, surfacing the manifold of value settings that solve a task rather than a single frozen compromise. Two qualifications, owed to the philosophical review. First, openness is only as wide as the expressible structures: a weighted sum is act consequentialism, and §5.7 shows that aggregator choice, side constraints, and theory uncertainty are distinct moral objects the framework must (and now does) represent rather than approximate with weights. Second, open specification *manages* the specification trap [21] procedurally rather than resolving it: the trap relocates from frozen weights to the operator's hand on the dial, and which hand that should be—justifiability to those affected, in the contractualist's sense [19]—is a question no architecture settles. The tuner's solving manifold is at best a crude contractualist surface: the set of configurations meeting every stakeholder's declared minimum.

Relationship to HAARF. For autonomous-agent governance frameworks such as HAARF [24], these mechanisms map onto verification and auditability controls: dynamics as reviewable artifacts

(code review of the simulator), declared invariants (pre-deployment safety constraints checked at synthesis time), dial settings logged per trajectory (value-configuration audit trails), and judge audits (second-model oversight with measured accuracy).

Limitations. (i) Scope: symbolic-state worlds; pixel-native domains remain the territory of learned models—the comparison here is within the symbolic regime, and neither the LLM proxy nor the numpy baselines is a trained Dreamer. The headline comparison also gives the synthesizer the rule text; E37 shows the structural OOD advantage survives removing that asymmetry (induction from traces alone), but induction is then imperfect (0.43 vs the rule-text 1.00), so part of the headline margin reflects the value of an explicit specification, not the representation alone. (ii) Complexity: monolithic synthesis with a 7B generator is reliable at four interacting rules and unreliable past roughly eight (E20); the headline worlds sit below that cliff, and richer worlds will need compositional synthesis [17] or larger generators. (iii) Scale: twenty repair tasks and three handwritten worlds; paired tests sharpen the comparisons, but the benchmark remains an archetype suite, not HumanEval [5], 3B+ agents saturate it, and the classic defect archetypes are plausibly present in pretraining data: agent solve rates should not be read as measurements of debugging skill—the world-model claims are unaffected, since the environment executes exactly regardless of what the agent has memorized. (iv) Synthesis replicates across three model families (E16), but the judge, critic, and repair-agent roles remain Qwen-only, and all experiments ran on one machine under the stated quantizations. (v) Judges: selection accuracy and order-consistency are measured but imperfect, the pooled selection margin is marginal ($p = 0.078$) and costs $\sim 4\times$ inference, and rubric grading showed compression; judge verdicts should gate, not replace, programmatic verification. (vi) Ensemble self-checking (E23) achieved perfect precision/recall here but is structurally vulnerable to correlated errors across generators reading the same ambiguous rule; it is a screen, not a proof. (vii) The sandbox is best-effort isolation against accident, not an adversarial security boundary (generated patches twice defeated in-process timeouts before the fork-based runner; see the repository history). (viii) The moral machinery, even broadened, evaluates declared values; whose values are declared—and the legitimacy of the declaring hand—is a procedural question outside any architecture [19], and the parliament's credences are themselves operator-set dials. (ix) Several experiments were redesigned after pilots and four internal review rounds (E3, E5/E6, E11–E27); pilot saturation results are reported alongside the final protocols, and all reviews ship in the repository.

7 Conclusion

A training-free symbolic world model—synthesized as code by a local model, verified before acceptance, steered by declared moral dials, optimized by an automated tuner, and assisted by an agent-as-a-judge—is bit-exact where learned-style dynamics compound error, transfers across $100\times$ scale shifts where trained baselines collapse, rolls out orders of magnitude faster, plans *as well as the ground truth itself* where planning through a hallucinating model is worse than no model, extends to stochastic dynamics with verified distributions, and flags its own synthesis errors by ensemble disagreement. Its boundary is equally measured: monolithic 7B synthesis fails past roughly eight interacting rules. The world-model field's trajectory has been to scale parameters until hallucination is suppressed; these results argue for an orthogonal path—make the model a program, verify the program, and keep the values dialable. Next steps: compositional synthesis to cross the complexity cliff [17], the coding world at standard-benchmark scale, hybrid perception-to-symbol pipelines, and judges that propose—not just select—world repairs.

Declaration of generative AI use

During the preparation of this work the author used *Claude Fable 5 (Anthropic, via Claude Code)* to implement the OpenWorld framework and experiment scripts, execute the experimental campaign, generate figures and tables, and draft this manuscript, under the author's direction. The author reviewed and edited the content and takes full responsibility for the content of the publication. All experimental results are produced by the released deterministic scripts (not by generative-AI estimation), and no AI tool is listed as an author, as such tools do not satisfy authorship criteria.

Data and code availability

All code, the benchmark suite, experiment scripts, raw result JSON, and this manuscript's build are available at `github.com/jim-schwoebel/openworld`. All data are synthetic; no human-subject or patient data were used.

Author contributions

J.S. conceived the study, directed the implementation and experiments, and reviewed the manuscript.

Competing interests

The author declares no competing interests.

Funding

This work received no specific external funding.

A Detailed result tables

Generator (family)	Runs	Verified acceptance (95% CI)	Probe acc. of accepted	Mean synthesis time
qwen2.5:7b (Qwen)	15	100% [0.80, 1.00]	0.98	—
qwen2.5:3b (Qwen)	15	100% [0.80, 1.00]	0.87	—
llama3.1:8b (Llama)	9	100% [0.70, 1.00]	0.85	9s
gemma2:9b (Gemma)	9	100% [0.70, 1.00]	1.00	4s

Table 10: E2 + E16: synthesis reliability by generator scale and family. Qwen rows: five compilation attempts per world; Llama/Gemma rows: three per world. Maximum four verify–repair iterations each; wall-clock includes all iterations.

Regime	Acceptance	Probe acc. (accepted)	Probe acc. (all runs)
Filter only (max_iters=1)	62%	0.62	0.39
Filter + repair loop (max_iters=4)	100%	0.65	0.65

Table 11: E18: filter-vs-repair decomposition of the full verification gate (3B generator, eight paired high-temperature attempts per regime).

Engine	1×	10×	100×
Synthesized code (0 transitions)	10/10	10/10	10/10
Delta-MLP, 128h (10k transitions)	5/10	0/10	0/10
MLP, 64h (10k transitions)	5/10	0/10	0/10
1-NN memorizer (10k transitions)	3/10	0/10	0/10
LLM next-state (7B, 0 transitions)	4/10	4/10	5/10

Table 12: E19: exact transition accuracy on the branch-covering probe suite across a 1×/10×/100× scale ladder.

Strategy	Seeds solving	Mean trials to first solve	Mean best score
random	10/10	4	17.60
tpe	10/10	5	17.60
random+refine	10/10	4	17.60

Table 13: E9: tuning-strategy comparison on the triage auto-configuration problem; ten seeds per strategy, 200-trial budget.

B The synthesized dynamics artifact

A representative accepted sprint-world transition, synthesized by `qwen2.5:7b` and verified by the full gate—the artifact whose exactness underwrites Table 2:

Accepted `transition()` (verbatim)

```
def transition(state, action):
    next_state = dict(state)
    name = action["name"]
    if name == "ship" and next_state["backlog"] > 0:
        next_state["backlog"] -= 1
        next_state["shipped"] += 1
        next_state["debt"] += 1
        next_state["bugs"] += next_state["debt"] // 4
    elif name == "fix":
        next_state["bugs"] = max(0, next_state["bugs"] - 2)
    elif name == "refactor":
        next_state["debt"] = max(0, next_state["debt"] - 2)
    return next_state
```

C Example tasks and failure cases

The coding-world tasks pair a buggy function with hidden tests; e.g. `dedupe` returns `list(set(xs))` (order-destroying) and must preserve first-seen order, and `balanced` counts parenthesis depth but misses early-imbalance strings like `")("`. Failure modes observed in the campaign: the 1.5B baseline resubmitted near-identical patches until the attempt budget expired on tasks where its first reading of the spec was wrong—precisely the local minimum that high-temperature candidate sampling plus judge selection escapes; conversely, when several candidates passed, the judge's pick was strongly order-sensitive (E13: raw choices matched under order reversal on only 28% of rounds) while its accuracy was not—evidence that judge criteria and presentation order need auditing even when outcomes look fine. In E7/E15, the rubric judge compressed mid-range episodes toward similar scores (rank correlation 0.75 rather than 1.0, dropping to 0.58 under paraphrase), consistent with

known LLM-judge coarseness [25].

D Reproducibility

One pipeline rebuilds every artifact: experiment scripts in `experiments/` write `results/*.json` (seeds fixed in-source); `python3 scripts/make_paper_assets.py` regenerates every figure, table, and the `numbers.tex` macros from those JSON files; and `make` in `paper/` builds this PDF. Model snapshots: `qwen2.5:7b` (Q4_K_M) and `qwen2.5:3b` via Ollama; platform recorded in each result file.

References

[1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.

[2] Eloi Alonso, Adam Jelley, Vincent Micheli, et al. Diffusion for world modeling: Visual details matter in Atari. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[3] Isaiah Berlin. Two concepts of liberty. In *Four Essays on Liberty*. Oxford University Press, 1969.

[4] Jake Bruce, Michael Dennis, Ashley Edwards, et al. Genie: Generative interactive environments. *arXiv preprint arXiv:2402.15391*, 2024.

[5] Mark Chen, Jerry Tworek, Heewoo Jun, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[6] Nicola Dainese, Matteo Merler, Minttu Alakuijala, and Pekka Marttinen. Generating code world models with large language models guided by Monte Carlo tree search. *arXiv preprint arXiv:2405.15383*, 2024.

[7] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.

[8] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

[9] Wolfgang Lehrach, Joseph Marino, Luis Piloto, et al. Code world models for general game playing. *arXiv preprint arXiv:2510.04542*, 2025.

[10] William MacAskill, Krister Bykvist, and Toby Ord. *Moral Uncertainty*. Oxford University Press, 2020.

[11] MAS Authors. Moral anchor system: A predictive framework for AI value alignment and drift prevention. *arXiv preprint arXiv:2510.04073*, 2025.

[12] Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. In *International Conference on Learning Representations (ICLR)*, 2023.

[13] Vincent Micheli, Eloi Alonso, and François Fleuret. Efficient world models with context-aware tokenization. *arXiv preprint arXiv:2406.19320*, 2024.

[14] NE-Dreamer Authors. Next embedding prediction makes world models stronger. *arXiv preprint arXiv:2603.02765*, 2026.

[15] NeSyS Authors. Neuro-symbolic synergy for interactive world modeling. *arXiv preprint arXiv:2602.10480*, 2026.

[16] Robert Nozick. *Anarchy, State, and Utopia*. Basic Books, 1974.

[17] Wasu Top Piriyakulkij, Yichao Liang, Hao Tang, et al. PoE-World: Compositional world modeling with products of programmatic experts. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[18] John Rawls. *A Theory of Justice*. Harvard University Press, 1971.

[19] Thomas M. Scanlon. *What We Owe to Each Other*. Harvard University Press, 1998.

[20] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, et al. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020.

[21] Specification Trap Authors. The specification trap: Why static value alignment alone is insufficient for robust alignment. *arXiv preprint arXiv:2512.03048*, 2025.

[22] Stable Value Alignment Authors. Toward stable value alignment: Introducing independent modules for consistent value guidance. *arXiv preprint arXiv:2605.11712*, 2026.

[23] Hao Tang, Darren Key, and Kevin Ellis. WorldCoder, a model-based LLM agent: Building world models by writing code and interacting with the environment. *arXiv preprint arXiv:2402.12275*, 2024.

[24] Task Force for AI Agents in Healthcare. HAARF: Healthcare AI agents regulatory framework — a comprehensive security verification standard for autonomous AI systems in clinical environments. *Task Force for AI Agents in Healthcare Technical Report*, 2026.

[25] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[26] Mingchen Zhuge, Changsheng Zhao, Dylan Ashley, et al. Agent-as-a-judge: Evaluate agents with agents. *arXiv preprint arXiv:2410.10934*, 2024.